Deep Neural Networks for Learning Intent from sEMG Signals to Support Hardware Devices for Post-Stroke Neurorehabilitation

Post-Stroke Neurorehabilitation Project Team

July 28, 2026

Abstract

Finger-specific motor intent is a clinically meaningful target for post-stroke neurorehabilitation, where residual muscle activity may be weak, variable, and patient-specific. This paper studies five-finger intent decoding from impaired-arm high-density surface electromyography (sEMG) using PhysioMio, a bilateral longitudinal dataset collected from stroke patients [9]. We formulate the task as multilabel prediction and evaluate recurrent, convolutional, and graph-based decoders under a common signal-processing protocol: label alignment, 20–450 Hz Butterworth filtering, Symlet-4 wavelet denoising, 200 ms overlapping windows, and twelve handcrafted time- and frequency-domain descriptors per channel-window pair. In the direct baseline comparison, the LSTM obtains the highest subset accuracy, also called exact match ratio (0.545), while the GNN obtains the strongest macro F1 (0.706) and macro AUPRC (0.776). We then evaluate optimized CNN students and 1D ResNet teachers as deployment-oriented alternatives for Raspberry Pi 5-supported rehabilitation hardware [17, 18]. CNN-Large gives the strongest single-split aggregate CNN result, with 0.593 subset accuracy, 0.798 finger accuracy, 0.714 macro F1, 0.881 macro AUROC, and 0.812 macro AUPRC. For hardware deployment, CNN-Micro is selected because it retains similar aggregate performance at 158K parameters and an estimated 154 KB INT8 footprint. Healthy-to-impaired transfer improves the CNN-Base branch but gives mixed LSTM results. Overall, the study shows that compact neural decoders can convert post-stroke impaired-arm sEMG into five-finger intent estimates while making model-size and deployment constraints explicit.

1 Introduction

Stroke frequently disrupts the descending motor pathways needed for coordinated hand opening, grasping, and individuated finger control. For patients with residual forearm muscle activity, surface electromyography (sEMG) offers a non-invasive sensing modality through which intended movement can be estimated even when visible motion is weak or incomplete [14, 22, 5]. Reliable finger-intent decoding is therefore a central software problem for rehabilitation devices that aim to provide timely assistance, feedback, or task-specific practice.

The decoding problem is difficult because post-stroke sEMG differs substantially from healthy-limb gesture-recognition benchmarks. Paretic recordings are affected by abnormal co-contraction, altered recruitment patterns, fatigue, sensor placement, and impairment severity. In addition, the intended output for rehabilitation is often not a single gesture class but a multi-finger activation pattern suitable for controlling or cueing assistive hardware. A useful model must therefore be evaluated together with the signal-processing and labeling decisions that define the task.

This paper studies that problem as a hardware-aware machine-learning question: how can impaired-arm HD-sEMG be transformed into five-finger intent estimates with models small enough to support responsive Raspberry Pi 5 rehabilitation hardware? The study evaluates the full decoding chain from label alignment and signal processing through model-family comparison, transfer learning, and compact CNN selection. This framing is deliberately tied to embedded rehabilitation use, because a clinically useful decoder must balance prediction quality with model size, latency, and deployability.

Our study is organized around two questions. First, when LSTM, CNN, and GNN decoders are trained on the same PhysioMio-derived feature representation, which inductive biases are most useful for multilabel finger-intent prediction? Second, after the baseline comparison, how much can decoding performance and deployment feasibility be improved through compact CNN architecture search, ResNet teacher references, and healthy-to-impaired transfer learning?

The central contribution is the formulation and evaluation of post-stroke HD-sEMG finger-intent decoding as a single hardware-aware research problem. We define a reproducible sEMG-to-finger-intent workflow for PhysioMio recordings, including label alignment, filtering, denoising, windowed feature extraction, patient-level splits, and shared tensor adaptation. We compare LSTM, CNN, and GNN predictors under this common formulation and report both aggregate and finger-wise multilabel metrics. We evaluate an optimized CNN student family against internal ResNet teachers and quantify the performance-size trade-off relevant to embedded deployment. Finally, we analyze healthy-to-impaired transfer learning and identify distillation and device-level latency measurement as the next compression and deployment experiments.

2 Related Work

sEMG has long been studied as a non-invasive control channel for rehabilitation, prosthetics, and human-machine interaction, and stroke-specific reviews show that sEMG-driven interventions can support meaningful upper-limb rehabilitation when signal interpretation is reliable enough for assistive use [14, 22]. Broader reviews of wearable sensing for stroke recovery make a related point: clinical value depends on the full stack, including acquisition quality, body placement, labeling protocol, feature representation, and the way machine learning is coupled to assessment or assistive control [5]. For this reason, decoding architecture should not be discussed independently of data construction and deployment context.

Dataset choice is especially important in this area. Ninapro remains the dominant public benchmark family for hand-gesture decoding from sEMG, with ten sub-datasets spanning different electrode layouts, movement vocabularies, and sensor modalities [3, 16]. The widely used DB5 split, for example, records 52 movements from 10 healthy participants with two Myo armbands, while DB1 and DB2 cover larger healthy cohorts with different channel configurations and richer kinematic metadata [3, 16]. By contrast, PhysioMio was introduced in 2026 as a stroke-specific bilateral and longitudinal HD-sEMG resource with 48 stroke patients, 16 gestures, 64 electrodes, and repeated recordings across inpatient recovery [9]. That distinction matters because results on healthy-subject multiclass gesture sets are not automatically transferable to impaired-arm intent decoding in post-stroke populations.

Stroke-specific decoding studies have historically been small and heterogeneous. Lee et al. [11] used subject-specific myoelectric pattern classification for six functional hand movements in chronic stroke survivors and reported a sharp impairment-dependent gap, with mean accuracy of 71.3% for moderately impaired participants and 37.9% for severely impaired participants. Anastasiev et al. [2] later studied post-acute stroke gesture recognition with portable eight-channel sEMG

and found that affected-side SVM performance could still reach the high-80% range on reduced gesture sets, but the task remained sensitive to feature design, label count, and paretic signal quality. More recent deep-learning work has begun to explore richer feature domains and architectures for post-stroke myoelectric recognition; Bao et al. [4] evaluated CNN, CNN-LSTM, and CNN-LSTM-attention designs across time, frequency, and wavelet features in chronic stroke participants, with the best reported intra-subject and inter-subject transfer settings reaching 72.95% and 68.38% average accuracy, respectively. These studies underscore a recurring challenge for post-stroke intent decoding: strong performance is possible, but dataset scale, subject condition, movement vocabulary, and train-test protocol strongly determine the meaning of any reported number.

Deep neural models broaden that picture but do not remove the comparability problem. Sequence-oriented recurrent models remain natural baselines because they aggregate temporal evidence across windows, while compact 1D CNNs emphasize local temporal motifs and can be compressed aggressively for efficient inference [8]. Graph neural networks offer a complementary structural view by representing windows as related observations connected through message passing rather than as a purely linear sequence [10, 6, 20]. On healthy-dataset benchmarks such as Ninapro, deeper end-to-end networks can reach very high multiclass accuracies; for example, Sri-Iesaranusorn et al. [19] reported 93.87% accuracy on Ninapro DB5 and 91.69% on DB7 for 41-movement classification. Other efficient CNN designs on Ninapro DB5 report high-80% accuracy while explicitly optimizing computation or channel use [15]. Those results are useful context for model capacity, but they address a different population and a different task than impaired-arm multilabel finger-intent prediction.

Transfer learning has recently become one of the most relevant directions for rehabilitation-oriented sEMG. Li et al. [12] showed that transfer-based CNNs can materially improve inter-subject and inter-day robustness on high-density healthy-subject sEMG, and Mohammadiazni et al. [13] demonstrated an especially important stroke-specific result: pretraining on healthy Ninapro DB5 data raised stroke-survivor gesture-classification accuracy from 46% to 93.6% in a three-gesture setting. These findings motivate healthy-to-impaired transfer as a natural strategy for paretic-limb decoding, where affected-side training data are comparatively scarce and variable.

Deployment-oriented optimization provides the final piece of context for this manuscript. Knowledge distillation is a standard way to transfer behavior from larger teachers into smaller students [7], and Optuna is widely used for black-box hyperparameter search under practical constraints [1]. For rehabilitation wearables, these methods connect offline decoding accuracy to the practical requirements of model size, latency, and hardware-constrained inference.

Table 1: Literature context for the current study. Reported numbers reflect different populations, sensors, gesture vocabularies, and label spaces.

Source	Population / dataset	Task	Reported performance	Relevance to this paper
Lee et al. 2010	20 chronic stroke survivors; 10 forearm/hand electrodes	Six functional hand movements with subject-specific EMG classification	Mean accuracy 71.3% for moderate impairment and 37.9% for severe impairment	Early evidence that stroke severity strongly affects decoder quality
Anastasiev et al. 2022	19 post-acute stroke patients; portable eight-channel sEMG	Four- to seven-gesture multiclass recognition on affected and non-affected sides	Affected-side SVM accuracy reached 88.7% on the four-gesture setting	Shows that reduced-vocabulary stroke gesture recognition can reach high accuracy with careful feature design
Bao et al. 2024	8 chronic stroke participants; post-stroke sEMG feature-domain study	CNN, CNN-LSTM, and CNN-LSTM-attention recognition using time, frequency, and wavelet features	Best reported averages: 72.95% intra-subject accuracy and 68.38% inter-subject transfer accuracy	Supports deep architectures for post-stroke decoding while highlighting the difficulty of transfer
Mohammadiazni et al. 2026	10 stroke patients plus healthy Ninapro DB5 pretraining data	Three-gesture classification across multiple arm postures with transfer learning	93.6% with transfer learning versus 46% without transfer	Strong stroke-specific motivation for healthy-to-impaired transfer learning
Sri-Iesaranusorn et al. 2021	Ninapro DB5/DB7, mostly healthy-subject settings	41-movement deep neural network classification	93.87% on DB5 and 91.69% on DB7	Healthy-data ceiling is high, but the task is easier than impaired-arm multilabel finger decoding
This project	PhysioMio impaired-arm split from 48 stroke patients; patient-level train/val/test split	Five-finger multilabel intent decoding from processed HD-sEMG segments	Offline leader: CNN-Large with 0.593 subset accuracy and 0.714 macro F1; deployment choice: CNN-Micro with 0.578 subset accuracy and 154 KB INT8 size	Stricter label space and patient split make this a harder but more rehabilitation-specific benchmark

3 Method

3.1 Overview

This project defines a modular decoding workflow from multichannel sEMG and gesture annotations to five-finger prediction logits. The pipeline contains four stages: data harmonization, signal processing, model inference, and evaluation or deployment analysis. Figure 1 summarizes the workflow used throughout the study.

For notation, let a pre-segmented raw recording be $X_{\text{raw}} \in \mathbb{R}^{T \times C}$, where T is the number of raw time samples and $C = 64$ is the channel count under the PhysioMio path. After preprocessing and windowing, the project extracts a feature tensor $X_{\text{feat}} \in \mathbb{R}^{C \times W \times F}$, where W is the number of windows and $F = 12$ is the number of handcrafted descriptors per channel-window pair. Each sample is paired with a multilabel target vector $y \in \{0, 1\}^5$ indicating intended activation of thumb, index, middle, ring, and little finger.

3.2 Data harmonization and ingestion

The implementation builds processed datasets directly from raw PhysioMio parquet files. The loader searches the raw data tree, groups files by patient, aligns contiguous stretches of the same `movement_type`, maps each label to a five-bit finger target, discards contiguous label segments shorter than 200 raw samples, and applies the preprocessing pipeline to each usable segment. At the default 2000 Hz sampling rate, this 200-sample rule is a permissive minimum-duration filter for label segments; feature extraction still requires complete 200 ms windows, corresponding to 400 raw samples at 2000 Hz, because the windowing stage does not zero-pad short segments. This converts gesture-level clinical recordings into segment-level multilabel examples.

The default configuration uses PhysioMio recordings sampled at 2000 Hz and selects the impaired arm. Patients are partitioned into deterministic 70/10/20 train/validation/test splits, so samples from the same patient do not appear in multiple partitions. Processed samples are padded to a dataset-wide maximum window count for batched tensor operations.

3.3 Signal preprocessing

The preprocessing module is explicitly parameterized by a frozen configuration object in the implementation. Raw sEMG first passes through a fourth-order Butterworth band-pass filter with 20–450 Hz cutoffs and zero-phase forward-backward application. This stage suppresses low-frequency motion artifacts and high-frequency electrical noise while preserving time alignment across channels.

The filtered signal is then denoised with wavelet soft-thresholding. The default configuration uses a Symlet-4 basis, four decomposition levels, universal threshold selection, and a median-based noise estimate. The same preprocessing configuration is used for all model families, making downstream differences attributable to the representation and predictor rather than to separate signal-conditioning choices.

After denoising, each continuous segment is divided into 200 ms windows with 50% overlap and no zero-padding. These settings balance temporal responsiveness against the sample count required for stable summary statistics.

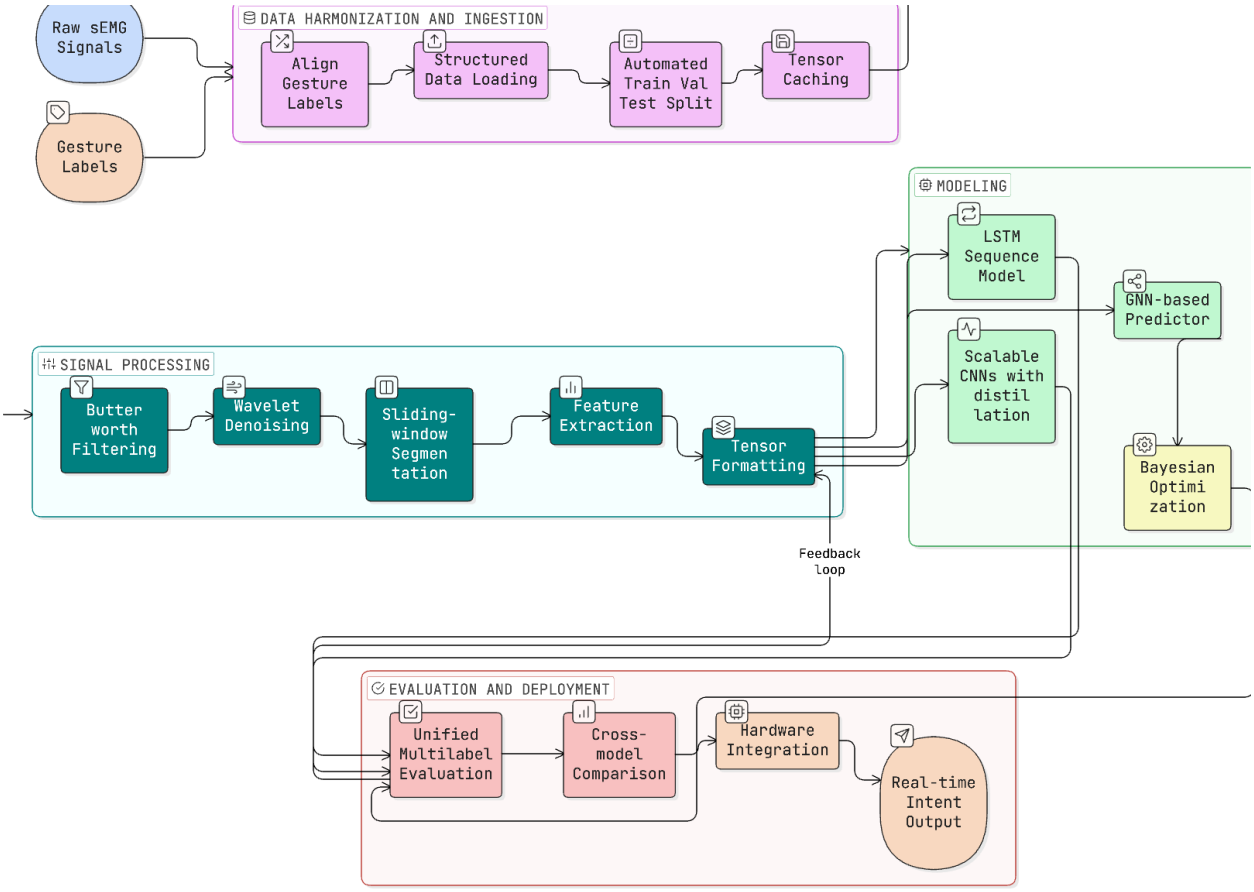

Figure 1: Overview of the sEMG finger-intent decoding pipeline. Raw sEMG and gesture annotations are aligned, segmented, featurized, mapped into shared tensors, evaluated across multiple model families, and prepared for downstream deployment analysis.

3.4 Window-level feature representation

Each channel-window pair is summarized by twelve handcrafted descriptors spanning time and frequency domains. The time-domain set includes root mean square, mean absolute value, integrated EMG, waveform length, variance, zero crossings, slope sign changes, and Willison amplitude. The frequency-domain set includes mean frequency, median frequency, spectral entropy, and total power, with spectral quantities estimated from Welch periodograms using `nperseg = min(256, len(x))` [21]. For the default 200 ms PhysioMio window, $\text{len}(x) = 400$, so the Welch segment length is 256 samples; shorter windows, if produced by alternate configurations, use their full length. Because no explicit `noverlap` is passed, SciPy uses its default 50% periodogram overlap. Degenerate spectra are zeroed explicitly to avoid unstable feature values. The resulting tensor has shape (C, W, F), where C is the number of channels, W is the number of windows, and $F = 12$ is the feature count.

This feature representation is a deliberate engineering choice rather than a claim that raw end-to-end learning is unnecessary. Handcrafted time- and frequency-domain summaries reduce the input dimensionality, expose standard sEMG descriptors to all model families, and support compact students intended for embedded inference. They are also appropriate for the current single-dataset

setting, where learning stable low-level time-frequency filters directly from raw HD-sEMG would require additional data and ablation experiments.

3.5 Tensor formatting and shared adapter

The project standardizes cross-model comparison with one shared adapter. It transforms (C, W, F) or (N, C, W, F) tensors into $(N, W, C \times F)$ sequences, where windows act as time steps and all channel features are concatenated into one vector per step. For a single sample, the adapted sequence can be written as

$$S = [s_1, \ldots, s_W]^\top \in \mathbb{R}^{W \times d}, \qquad s_w = \mathrm{vec}(X_{\mathrm{feat},:,w,:}), \qquad d = CF = 768.$$

This adapter creates a common input space for recurrent, convolutional, and graph-based predictors.

For the CNN branch, the adapted tensor is subsequently permuted to a channels-first layout. The CNN stack treats the 64×12 channel-feature grid as 768 input channels for temporal `Conv1d` layers, preserving the same upstream representation while allowing convolutions over window order. This is best interpreted as temporal convolution over a feature sequence whose per-window vector already contains spatial-channel and spectral summaries. It is not asserted to be superior to raw-signal 1D CNNs or 2D channel-by-time convolutions; those alternatives remain important ablations for future work.

3.6 Predictor families

The LSTM branch applies two stacked recurrent layers with hidden sizes 128 and 256, followed by a 128-unit fully connected layer and dropout before five output logits. Given adapted sequence input s_t, the recurrent dynamics follow the standard LSTM update

$$\begin{aligned}
i_t &= \sigma(W_i s_t + U_i h_{t-1} + b_i),\\
f_t &= \sigma(W_f s_t + U_f h_{t-1} + b_f),\\
o_t &= \sigma(W_o s_t + U_o h_{t-1} + b_o),\\
\tilde{c}_t &= \tanh(W_c s_t + U_c h_{t-1} + b_c),\\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,\\
h_t &= o_t \odot \tanh(c_t),
\end{aligned}$$

and the project uses the final hidden state from the second recurrent layer for classification. This branch emphasizes temporal accumulation across windows and serves as the main sequential baseline for the direct benchmark.

The GNN branch interprets each time window as a node in a graph. The implementation supports GCN, GraphSAGE, and GAT operators [10, 6, 20]; the benchmarked direct model uses the default two-layer GCN setting with hidden dimension 64, ReLU activations between graph-convolution layers, dropout 0.5, and mean graph pooling. The benchmarked path constructs a complete undirected graph over windows within each sample before graph-level pooling and readout. The resulting edge set has $O(W^2)$ edges per sample, which is computationally feasible for the short padded window sequences used here and lets each window exchange information with all other windows. Sparse temporal adjacency and k-nearest-neighbor graph construction are left as graph-design ablations. At a generic level, the node update can be written as

$$h_i^{(\ell+1)} = \psi_\ell\left(h_i^{(\ell)}, \underset{j \in \mathcal{N}(i)}{\mathrm{AGG}} \phi_\ell\left(h_i^{(\ell)}, h_j^{(\ell)}\right)\right),$$

followed by graph pooling

$$g = \mathrm{POOL}\Big(\{h_i^{(L)}\}_{i=1}^{W}\Big)$$

and a multilabel readout head. In the benchmarked GCN setting, AGG is the normalized neighborhood aggregation implemented by `GCNConv`, ψ_ℓ is a linear graph-convolution update followed by ReLU for non-final layers, and `POOL` is global mean pooling. This branch injects an explicit structural prior: windows are related observations rather than only a flat sequence.

The CNN branch includes five student variants ranging from Nano to XLarge, each built around a 1×1 projection, temporal convolution blocks, adaptive pooling, and a compact fully connected head. If the channels-first representation is denoted by $\hat{S} \in \mathbb{R}^{d \times W}$, the student stack can be summarized as

$$H^{(0)} = \rho\Big(\mathrm{BN}(W_p * \hat{S} + b_p)\Big), \qquad H^{(\ell+1)} = \rho\Big(\mathrm{BN}(W_\ell * H^{(\ell)} + b_\ell)\Big),$$

where $*$ denotes 1D convolution over window order and ρ is a ReLU nonlinearity. Adaptive pooling compresses the temporal axis before a multilayer perceptron produces the five logits. The student family spans approximately 54K parameters and 53 KB INT8 for Nano to 1.62M parameters and 1.58 MB INT8 for XLarge. The teacher family uses 1D ResNet-50, ResNet-101, and ResNet-152 variants with bottleneck residual blocks.

3.7 Training, tuning, and evaluation protocol

The training utilities unify optimization and reporting across model families. The current implementation uses binary cross-entropy with logits for five-label prediction, Adam-based optimization, plateau-triggered learning-rate reduction, early stopping, checkpointing, and saved metric curves. For logits $z \in \mathbb{R}^5$, the base multilabel objective is

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{5}\sum_{k=1}^{5}\left[y_k \log \sigma(z_k) + (1 - y_k)\log\left(1 - \sigma(z_k)\right)\right].$$

Evaluation reports subset accuracy, also called exact match ratio, finger-level averaged accuracy, macro precision, macro recall, macro F1, macro AUROC, and macro AUPRC. Subset accuracy counts a sample as correct only when all five finger labels are predicted correctly. The metrics module also stores per-finger scores and task-level curve data, which enables both aggregate and finger-wise analysis after training.

For the CNN students, Optuna searches architecture and training choices for each model size [1]. For a trial with validation macro F1 F_{val} reported on a 0–100 scale, measured latency t, and size-specific baseline latency t_{ref} measured after one baseline epoch, the objective is

$$\mathrm{score} = \frac{F_{\mathrm{val}}}{100}\, p\left(\frac{t}{t_{\mathrm{ref}}}\right),$$

where

$$p(r) = \begin{cases} 1, & r \le 1, \\ 1 - 3(r-1), & 1 < r \le 1.2, \\ 0.05, & r > 1.2. \end{cases}$$

Thus trials at or below the baseline latency are not penalized, mildly slower trials receive a linear penalty, and substantially slower trials are strongly downweighted. Trial-level early stopping and median pruning are used during search.

The system also implements teacher-ensemble distillation, per-finger threshold tuning, and latency benchmarking, but the present Results section reports only the non-distilled student and teacher benchmarks. For multilabel distillation, the implemented loss blends the hard target term with a temperature-scaled soft target from the teacher,

$$\mathcal{L}_{\mathrm{KD}} = \alpha\, \mathcal{L}_{\mathrm{BCE}}(z_s, y) + (1-\alpha) T^2\, \mathrm{BCELogits}\left(\frac{z_s}{T}, \sigma\left(\frac{z_t}{T}\right)\right),$$

where z_s and z_t are student and teacher logits, T is the distillation temperature, and α controls the hard-soft balance. A separate latency benchmark reports mean, median, and 95th-percentile inference time on a chosen device.

3.8 Transfer-learning path

In addition to direct impaired-arm benchmarks, the study evaluates two-stage CNN and LSTM training. Models are first pretrained on healthy-arm recordings and then finetuned on impaired-arm recordings while preserving the same five-finger multilabel objective. The reported CNN transfer experiment uses the CNN-Base configuration selected by the two-stage tuning run, so its pretraining and finetuning rows are not intended to duplicate either the legacy direct CNN baseline or the optimized CNN-Micro/CNN-Large deployment rows. This tests whether healthy-limb structure can provide a useful initialization for paretic-limb decoding.

4 Experiments

4.1 Dataset and split construction

All experiments use the PhysioMio processing path implemented for this project [9, 17]. The dataset construction step extracts contiguous movement segments from raw parquet recordings, maps gesture labels to five-bit finger targets, preprocesses each segment, and stores processed tensors as PyTorch split files.

Patients are shuffled with a fixed seed and divided into 70% train, 10% validation, and 20% test partitions. The main benchmark uses impaired-arm recordings. The transfer-learning experiments use healthy-arm recordings for pretraining and impaired-arm recordings for finetuning.

4.2 Model groups

The experiments are organized into three groups. The first group compares direct LSTM, CNN, and GNN baselines trained under the same preprocessing and feature representation. The second group evaluates optimized CNN students and ResNet teachers, emphasizing the trade-off between multilabel performance and model footprint. The third group evaluates healthy-to-impaired transfer learning. Distillation is implemented in the codebase as a teacher-student compression path, but it is not reported as a completed benchmark in the current results.

4.3 Evaluation protocol

The task is five-label multilabel classification with a nominal decision threshold of 0.5 on the held-out test split. We report subset accuracy, also called exact match ratio, mean finger accuracy, macro

precision, macro recall, macro F1, macro AUROC, macro AUPRC, and per-finger metrics. Subset accuracy is strict because all five labels must be correct for a sample to count as correct; finger accuracy and macro metrics expose behavior that subset accuracy alone can hide. The present tables report single deterministic train/validation/test split estimates from committed artifacts, so small differences should be interpreted as empirical comparisons rather than statistical significance claims.

The Results section follows the experimental sequence: direct baselines, optimized CNN students and ResNet teachers, transfer learning, finger-level behavior, and literature positioning.

5 Results

5.1 Direct baseline models: LSTM, CNN, and GNN

Table 2 reports the direct comparison among LSTM, CNN, and GNN decoders on the impaired-arm test split. The three families express different assumptions about the same windowed feature sequence. The LSTM treats the input as an ordered temporal process, the CNN searches for local temporal motifs after channel-feature projection, and the GNN treats windows as nodes in a sample-level graph. Their results show complementary behavior on this split: the LSTM leads subset accuracy and macro AUROC, while the GNN leads macro F1 and macro AUPRC.

Table 2: Held-out test performance for the three direct model families. Subset accuracy is the multilabel exact match ratio. Bold numbers mark the best score in each column within the baseline comparison.

Model	Subset Acc.	Finger Acc.	Macro F1	Macro AUROC	Macro AUPRC
LSTM	**0.545**	**0.784**	0.705	**0.858**	0.754
CNN legacy	0.442	0.694	0.676	0.767	0.764
GNN	0.448	0.683	**0.706**	0.787	**0.776**

The baseline comparison suggests that temporal recurrence and graph aggregation capture different aspects of paretic sEMG. The LSTM's stronger subset accuracy indicates better all-label consistency, whereas the GNN's macro F1 and AUPRC suggest stronger class-balanced discrimination despite lower finger accuracy. This pattern means the GNN ranks positive finger activations well across classes but does not uniformly improve the fixed-threshold decision for every finger. The original CNN is weaker in this stage, motivating a more systematic convolutional architecture search.

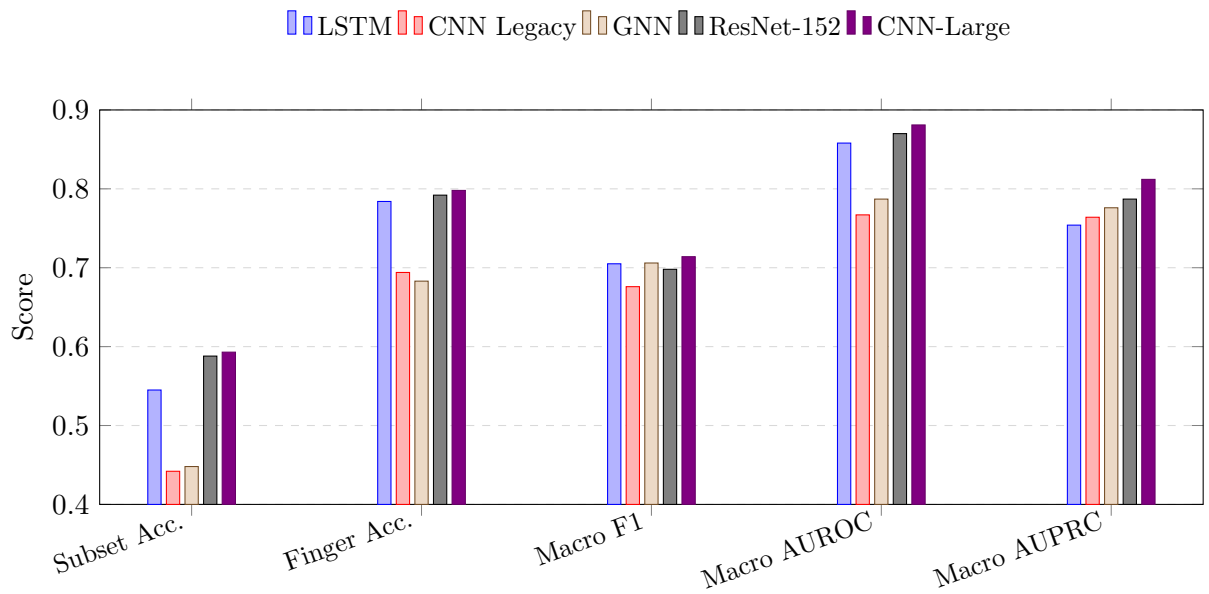

Figure 2: Aggregate-metric comparison across the three direct baselines plus the strongest internal teacher and student references from the newer CNN branch.

5.2 Performance-improvement path: optimized CNN students and ResNet teachers

Table 3 evaluates the optimized CNN student family together with the internal ResNet teacher models. Relative to the direct CNN baseline, the optimized students improve every major aggregate metric. The strongest accuracy model in this single-split evaluation is CNN-Large, which reaches 0.593 subset accuracy, 0.798 finger accuracy, 0.714 macro F1, 0.881 macro AUROC, and 0.812 macro AUPRC. The selected hardware-deployment model is CNN-Micro: at 158K parameters and an estimated 154 KB INT8 size, it retains 0.578 subset accuracy, 0.794 finger accuracy, 0.697 macro F1, 0.871 macro AUROC, and 0.793 macro AUPRC.

Table 3: CNN-family evaluation. Student parameter counts and INT8 sizes describe the deployment-oriented model scale; ResNet rows provide high-capacity teacher references. Bold numbers mark the best score in each column; † marks the selected hardware-deployment model.

Variant	Type	Params	INT8 Size	Subset Acc.	Finger Acc.	Macro F1	Macro AUROC	Macro AUPRC
ResNet-50	Teacher	–	–	0.570	0.788	0.656	0.840	0.772
ResNet-101	Teacher	–	–	0.569	0.795	0.681	0.852	0.767
ResNet-152	Teacher	–	–	0.588	0.792	0.698	0.870	0.787
Nano	Student	54K	53 KB	0.580	0.794	0.698	0.855	0.758
Micro†	Student	158K	154 KB	0.578	0.794	0.697	0.871	0.793
Base	Student	390K	381 KB	0.575	**0.798**	0.707	0.870	0.775
Large	Student	803K	784 KB	**0.593**	**0.798**	**0.714**	**0.881**	**0.812**
XLarge	Student	1.62M	1.58 MB	0.551	0.794	0.690	0.830	0.726

The optimized CNN results reveal a useful performance-size structure. CNN-Large improves subset accuracy from 0.442 to 0.593 relative to the direct CNN baseline, while macro AUROC rises from 0.767 to 0.881 and macro AUPRC from 0.764 to 0.812. CNN-Micro is within 0.015 subset accuracy, 0.017 macro F1, 0.011 macro AUROC, and 0.019 macro AUPRC of CNN-Large while

using approximately one fifth of the INT8 footprint. It also provides a stronger ranking profile than CNN-Nano, especially on macro AUPRC, and is therefore the best deployment compromise even though CNN-Large remains the offline accuracy leader. The sweep is not monotonic in parameter count: CNN-XLarge has the largest footprint but lower macro AUROC and macro AUPRC than CNN-Large. This suggests that the current dataset size, search budget, and regularization choices favor an intermediate-capacity temporal CNN rather than the largest available student. ResNet-152 is the strongest teacher with 0.588 subset accuracy and 0.698 macro F1; CNN-Large has higher single-split aggregate values on subset accuracy, macro F1, macro AUROC, and macro AUPRC, while CNN-Micro remains competitive with the ResNet line at a compact student scale.

5.3 Improvement paths for metrics and speed

Table 4 summarizes healthy-to-impaired transfer learning. This experiment uses the two-stage CNN-Base configuration selected during transfer tuning; it is separate from the legacy direct CNN baseline in Table 2 and from the optimized CNN-Micro/CNN-Large student sweep in Table 3. For CNN-Base, finetuning improves subset accuracy from 0.465 to 0.585, finger accuracy from 0.760 to 0.800, macro AUROC from 0.833 to 0.863, and macro AUPRC from 0.699 to 0.767. The LSTM branch behaves differently: subset accuracy and finger accuracy improve after finetuning, but macro F1, AUROC, and AUPRC decrease. Transfer learning therefore appears beneficial for the convolutional branch while showing architecture-dependent sensitivity in the recurrent branch.

Table 4: Healthy-to-impaired transfer-learning summaries. CNN rows use the tuned CNN-Base two-stage configuration. The CNN branch improves consistently after finetuning; the LSTM branch improves on subset and finger accuracy but declines on ranking-sensitive metrics.

Model / Stage	Subset Acc.	Finger Acc.	Macro F1	Macro AUROC	Macro AUPRC
CNN-Base pre.	0.465	0.760	0.664	0.833	0.699
CNN-Base fine.	**0.585**	**0.800**	**0.680**	**0.863**	**0.767**
LSTM pre.	0.523	0.752	**0.679**	**0.858**	**0.773**
LSTM fine.	**0.574**	**0.777**	0.660	0.830	0.736

The teacher-student design also provides a path toward faster inference, but it is not a completed benchmark result in this paper. The implemented distillation objective combines hard multilabel supervision with temperature-scaled teacher probabilities, allowing compact students to learn from ResNet teachers while retaining the same five-finger output space. Because the present study reports the optimized student and teacher evaluations separately, distillation is treated as the next compression experiment rather than as the source of the student results in Table 3.

5.4 Finger-level behavior and external positioning

Finger-wise scores show that model ranking varies across outputs. Table 5 compares the LSTM direct baseline, the GNN direct baseline, and the best optimized CNN student. The GNN is strongest on thumb, CNN-Large leads on index and little finger, and the LSTM remains slightly strongest on middle and ring finger. These patterns are plausible given the biomechanics of hand control, but they should be interpreted as hypotheses rather than anatomical proof. Thumb activation is more anatomically distinct from the ulnar fingers, so a graph model that exchanges information across all windows may better separate its activation pattern. Middle and ring fingers often appear in coupled grasping synergies, which may explain why the recurrent model's temporal state remains

useful for those outputs. CNN-Large's index and little-finger results suggest that local temporal motifs in the engineered feature sequence can support fingers that may appear as more discrete activation changes. These differences are clinically relevant because a rehabilitation controller may value different fingers differently depending on the movement being assisted.

Table 5: Per-finger F1 score by model family. The optimized CNN-Large replaces the legacy direct CNN row for the finger-level comparison. Bold numbers mark the strongest model for each finger.

Finger	LSTM	GNN	CNN-Large
Thumb	0.716	**0.776**	0.729
Index	0.723	0.696	**0.750**
Middle	**0.712**	0.689	0.698
Ring	**0.696**	0.688	0.695
Little	0.680	0.678	**0.696**

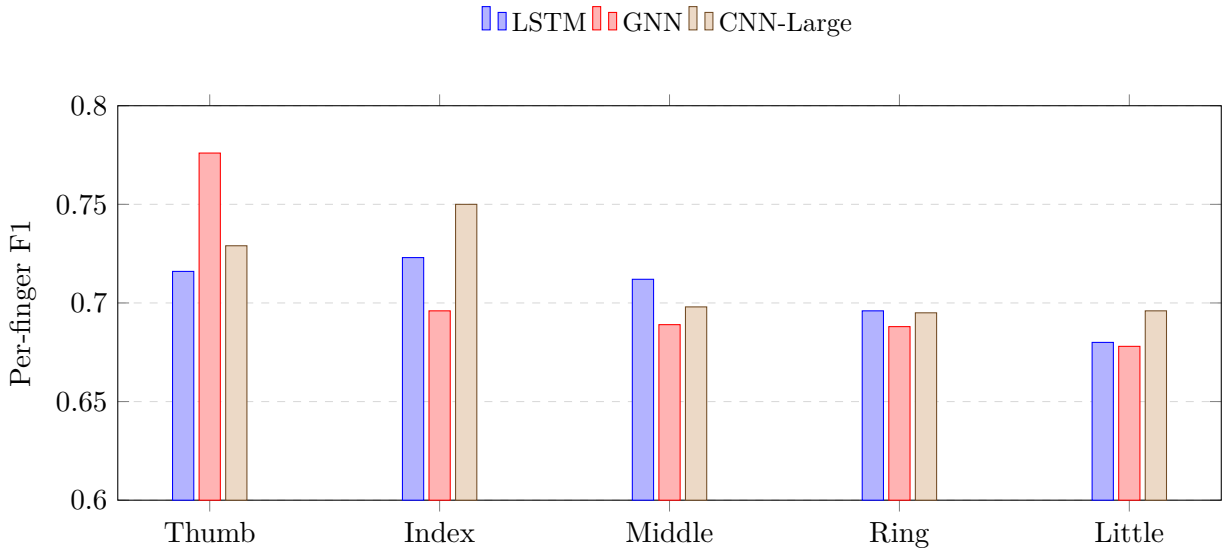

Figure 3: Per-finger F1 comparison derived from the committed metric files. Aggregate CNN gains coexist with output-specific strengths for the LSTM and GNN baselines.

Table 1 places the results beside prior stroke and sEMG studies. The comparison is not a leaderboard because the tasks differ substantially across datasets and populations. The present task is stricter than many gesture-classification settings: it uses impaired-arm PhysioMio recordings, patient-level splits, and a five-label subset-accuracy metric that requires all finger activations to be correct simultaneously. Within that setting, the optimized CNN branch is competitive with strong ResNet teachers and provides a practical deployment candidate in CNN-Micro, while the baseline and per-finger analyses show why multiple inductive biases remain scientifically informative.

6 Discussion

The results support three main observations about post-stroke finger-intent decoding from HD-sEMG. First, no single inductive bias is uniformly superior in the direct baseline setting. The LSTM's advantage on subset accuracy suggests that recurrent state can help produce coherent multi-finger

predictions across a segment, while the GNN's macro F1 and AUPRC indicate strong class-balanced discrimination when windows are modeled as related observations. This is consistent with the structure of the task: paretic sEMG contains both temporal continuity and non-local relationships among windows.

Second, the optimized CNN results show that compact temporal convolution can be effective once the architecture is tuned for the feature representation. The direct CNN baseline is weak, but the tuned student sweep shows that this is not a limitation of temporal convolution as a model family. CNN-Large gives the strongest offline aggregate CNN result in the current single-split experiments, but CNN-Micro is the deployment model chosen for the Raspberry Pi 5 pathway because it keeps the accuracy and ranking metrics close to CNN-Large while reducing the estimated INT8 footprint from 784 KB to 154 KB. The poorer CNN-XLarge result reinforces the same point from the opposite direction: increasing capacity alone is not enough, and the useful operating point is determined by the interaction among data scale, regularization, search budget, and deployment constraints.

Third, transfer learning appears useful but architecture-dependent. The CNN-Base branch benefits consistently from healthy-to-impaired staging, improving subset accuracy, finger accuracy, macro AUROC, and macro AUPRC. The LSTM branch improves on thresholded accuracy but loses ranking-sensitive performance after finetuning. This pattern aligns with recent stroke-specific transfer-learning work that uses healthy sEMG to compensate for limited paretic data [13], while also showing that transfer must be evaluated separately for each model family.

The external relevance of these results follows from the task definition. Healthy-subject Ninapro studies and reduced-vocabulary stroke studies often report higher multiclass accuracies under easier or different protocols, but those settings do not evaluate the same impaired-arm, patient-split, five-label PhysioMio problem. In this rehabilitation-specific setting, the optimized CNN family is competitive with high-capacity ResNet teachers, CNN-Large gives higher single-split values than the strongest ResNet teacher on the main aggregate metrics, and CNN-Micro offers the best hardware-aware operating point. The scientific value of the study is the combination of impaired-arm HD-sEMG processing, multilabel finger-intent formulation, three model families, ResNet teacher references, transfer-learning analysis, and compact deployment selection in one reproducible experimental stack.

The deployment implications are encouraging. CNN-Micro is small enough to motivate Raspberry Pi 5 inference, and the implemented distillation objective creates a principled mechanism for transferring teacher behavior into compact models. Because distilled students are not benchmarked in the present Results section, distillation should be read as a future compression experiment rather than as an established source of the reported CNN-Micro performance. The next scientific step is to measure the full chain under device-level timing constraints and to determine whether distilled students can improve CNN-Micro without sacrificing its footprint and latency advantages.

The study has several limitations. It uses one primary dataset, one deterministic patient-level split, and single-point metric estimates from the committed experiment artifacts; additional random seeds, confidence intervals, and patient-level cross-validation are needed before treating small metric gaps as statistically reliable. Additional ablations are also needed to isolate the effects of filtering, denoising, feature selection, graph construction, threshold choice, raw-signal learning, and 2D spatial-temporal convolution. Literature comparisons remain contextual because published sEMG studies vary widely in subject population, electrode layout, gesture vocabulary, and label formulation. Finally, the present results evaluate decoding performance, not therapeutic outcome; human-subject studies with hardware-in-the-loop feedback will be required to connect predicted intent to rehabilitation benefit.

7 Conclusion

This paper studied five-finger intent decoding from post-stroke HD-sEMG using a shared signal-processing and tensor-adaptation workflow. Direct LSTM, CNN, and GNN baselines showed complementary strengths: the LSTM gave the strongest subset accuracy, while the GNN gave the strongest macro F1 and AUPRC. The optimized CNN branch then provided the clearest deployment path. CNN-Large gave the strongest offline aggregate CNN performance, but CNN-Micro was selected for hardware deployment because it preserved most of that performance with 158K parameters and an estimated 154 KB INT8 footprint.

For rehabilitation engineers, the take-home message is that compact CNNs are strong candidates for embedded post-stroke finger-intent decoding, but recurrent and graph-based models should not be dismissed because they reveal different aspects of multi-finger behavior. The results support treating signal processing, model family, transfer strategy, and deployment constraints as one integrated design problem for responsive Raspberry Pi-class rehabilitation hardware.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[2] Alexey Anastasiev, Hideki Kadone, Aiki Marushima, Hiroki Watanabe, Alexander Zaboronok, Shinya Watanabe, Akira Matsumura, Kenji Suzuki, Yuji Matsumaru, and Eiichi Ishikawa. Supervised myoelectrical hand gesture recognition in post-acute stroke patients with upper limb paresis on affected and non-affected sides. *Sensors*, 22(22):8733, 2022. doi: 10.3390/s22228733.

[3] Manfredo Atzori, Arjan Gijsberts, Claudio Castellini, Barbara Caputo, Anne-Gabrielle Mittaz Hager, Simone Elsig, Giorgio Giatsidis, Franco Bassetto, and Henning Müller. Electromyography data for non-invasive naturally-controlled robotic hand prostheses. *Scientific Data*, 1:140053, 2014. doi: 10.1038/sdata.2014.53.

[4] Tianzhe Bao, Zhiyuan Lu, and Ping Zhou. Deep learning based post-stroke myoelectric gesture recognition: From feature construction to network design. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 2024. doi: 10.1109/TNSRE.2024.3521583. Early access.

[5] Issam Boukhennoufa, Xiaojun Zhai, Victor Utti, Jo Jackson, and Klaus McDonald-Maier. Wearable sensors and machine learning in post-stroke rehabilitation assessment: A systematic review. *Biomedical Signal Processing and Control*, 71:103197, 2022. doi: 10.1016/j.bspc.2021.103197.

[6] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017.

[7] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[8] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

[9] Julian Ilg, Alexander C. R. Oldemeier, Marie Fieweger, Luca Deuschel, Peter Rieckmann, Peter Young, Sabine Krause, and Tim C. Lueth. Physiomio: Bilateral and longitudinal hd-semg dataset of 16 hand gestures from 48 stroke patients. *Scientific Data*, 13(1):19, 2026. doi: 10.1038/s41597-026-06557-0.

[10] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[11] Sang Wook Lee, Kristin Wilson, Blair A. Lock, and Derek G. Kamper. Subject-specific myoelectric pattern classification of functional hand movements for stroke survivors. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 19(5):558–566, 2010. doi: 10.1109/TNSRE.2010.2079334.

[12] Jianfeng Li, Xinyu Jiang, Jiahao Fan, Yanjuan Geng, Fumin Jia, and Chenyun Dai. Deep end-to-end transfer learning for robust inter-subject and inter-day hand gesture recognition using surface emg. *Biomedical Signal Processing and Control*, 100:106892, 2025. doi: 10.1016/j.bspc.2024.106892.

[13] M. Mohammadiazni, K. Huszar, S. Peters, and A. L. Trejos. Hand gesture intention detection using semg and transfer learning in stroke survivors. *IEEE Journal of Biomedical and Health Informatics*, 2026. doi: 10.1109/JBHI.2026.3693109. Early access, published May 13, 2026.

[14] Maria Munoz-Novoa, Morten B. Kristoffersen, Katharina S. Sunnerhagen, Autumn Naber, Margit Alt Murphy, and Max Ortiz-Catalan. Upper limb stroke rehabilitation using surface electromyography: A systematic review and meta-analysis. *Frontiers in Human Neuroscience*, 16:897870, 2022. doi: 10.3389/fnhum.2022.897870.

[15] Sike Ni, Mohammed A. A. Al-qaness, Ammar Hawbani, Dalal Al-Alimi, Mohamed E. Abd Elaziz, and Ahmed A. Ewees. A survey on hand gesture recognition based on surface electromyography: Fundamentals, methods, applications, challenges and future trends. *Applied Soft Computing*, 166:112235, 2024. doi: 10.1016/j.asoc.2024.112235.

[16] Ninapro Project. The Non-Invasive Adaptive Prosthetics (Ninapro) Database. https://ninapro.hevs.ch/, 2026. Project site, accessed July 28, 2026.

[17] Post-Stroke Neurorehabilitation Project Team. sEMG Finger-Intent Decoding Software for Post-Stroke Neurorehabilitation. https://github.com/post-stroke-rehab/psr-pipeline, 2026. Project software implementation, accessed July 28, 2026.

[18] Raspberry Pi Ltd. Introducing: Raspberry Pi 5! https://www.raspberrypi.com/news/introducing-raspberry-pi-5/, 2023. Official product announcement, accessed July 28, 2026.

[19] Panyawut Sri-Iesaranusorn, Attawit Chaiyaroj, Chatchai Buekban, Songphon Dumnin, Ronachai Pongthornseri, Chusak Thanawattano, and Decho Surangsrirat. Classification of 41 hand and wrist movements via surface electromyogram using deep neural network. *Frontiers in Bioengineering and Biotechnology*, 9:548357, 2021. doi: 10.3389/fbioe.2021.548357.

[20] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

[21] Peter D. Welch. The use of fast fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.

[22] Mingde Zheng, Michael S. Crouch, and Michael S. Eggleston. Surface electromyography as a natural human-machine interface: A review. *arXiv preprint arXiv:2101.04658*, 2021.